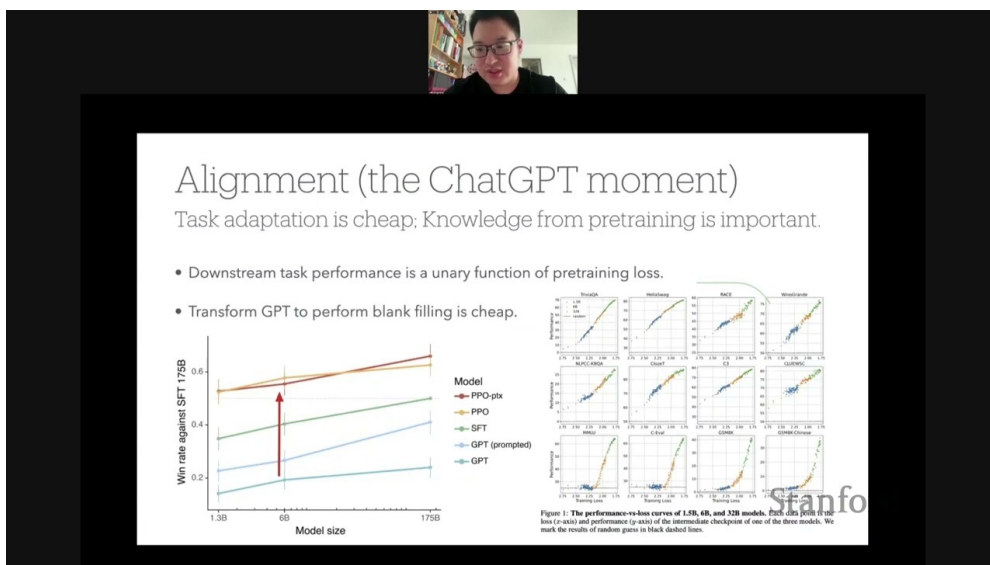


课程笔记

[CS25] From LLMs to Multimodal Models —Ming Ding, Zhipu AI

基于 Ming Ding 授课内容整理

2026 年 04 月 04 日



视频作者/频道: Stanford Online

发布日期: 2024 年 05 月 30 日

视频时长: 1 小时 20 分钟

视频链接: <https://www.youtube.com/watch?v=cYfKQ6YG9Qo>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：多模态大爆发的背景	3
1.1 演讲动机与布局	3
1.2 封面与视觉提示	3
1.3 本章小结	3
2 大语言模型的演进与基础	3
2.1 LLM 的三大驱动力	3
2.2 LLM 的治理与安全	4
2.3 LLM 演进时间线	4
2.4 本章小结	4
3 Mixtral 稠密基线与 Router 调度	5
3.1 Mixtral 的稠密基线	5
3.2 Router 负载均衡与专家调度	5
3.3 训练 Recipe: Mix 训练与阶段化	5
3.4 本章小结	6
4 视觉-语言模型的融合架构	6
4.1 架构范式梳理	6
4.2 CogVLM 的深度融合	6
4.3 GLM-4V 与实用化	7
4.4 本章小结	7
5 CogAgent：界面智能体与行动语言	7
5.1 输入-输出流水线	7
5.2 指导策略与模仿	7
5.3 本章小结	8
6 多模态生成：图像与视频	8
6.1 CogView 系列的演进	8
6.2 CogVideo 的时间一致性	8
6.3 融合趋势：自回归 + 扩散	8
6.4 本章小结	9
7 多模态数据平台与标注	9
7.1 数据采集与流水线	9
7.2 对齐学习与 pseudo label	9
7.3 本章小结	10
8 评估与多模态指标体系	10
8.1 质量 vs 成本 vs 可控性	10
8.2 Success Detector 与 Observability	10
8.3 本章小结	11

9 部署与观测：运行时的工程闭环	11
9.1 量化与推理流水线	11
9.2 观测雷达	11
9.3 本章小结	12
10 实践策略与未来演进	12
10.1 数据与模型的协同	12
10.2 未来方向	13
10.3 本章小结	13
11 案例复盘与经验	13
11.1 CogAgent in Production	13
11.2 CogVideo 的评测对比	13
11.3 本章小结	14
12 工程化策略与案例对比	14
12.1 Mixtral vs 传统 MoE Baseline	14
12.2 行业部署对比	14
12.3 故障演练与 Runbook	14
12.4 本章小结	15
13 合规与治理	15
13.1 数据合规与隐私	15
13.2 道德使用与防滥用	15
13.3 本章小结	15
14 局限与待解的挑战	16
14.1 对齐与 hallucination 仍未封顶	16
14.2 观测与长期迭代的矛盾	16
14.3 生态与开源的复杂性	16
14.4 本章小结	16
15 工具链、API 与自动化	16
15.1 训练与导出流水线	16
15.2 Inference API 与集成	17
15.3 本章小结	17
16 开源、合作与生态	17
16.1 Open Source 与社区贡献	17
16.2 合作与行业应用	17
16.3 本章小结	18
17 总结与延伸	18
17.1 层级总结表	18
17.2 拓展阅读	18
17.3 本章小结	18

1 引言：多模态大爆发的背景

1.1 演讲动机与布局

Ming Ding 在 Stanford CS25 的第 31 讲站在“LLM 时代的回望”与“多模态时代的展望”之间。他将讲稿分为三个层次：1) LLM 的历史与工程动能；2) 视觉、视频、多模态模型的架构实践；3) 部署/评估/观测的工程闭环。条理清晰，帮助听众从核心技术、数据与工程三个维度循序渐进地理解 Zhipu AI 的路线图。

1.2 封面与视觉提示

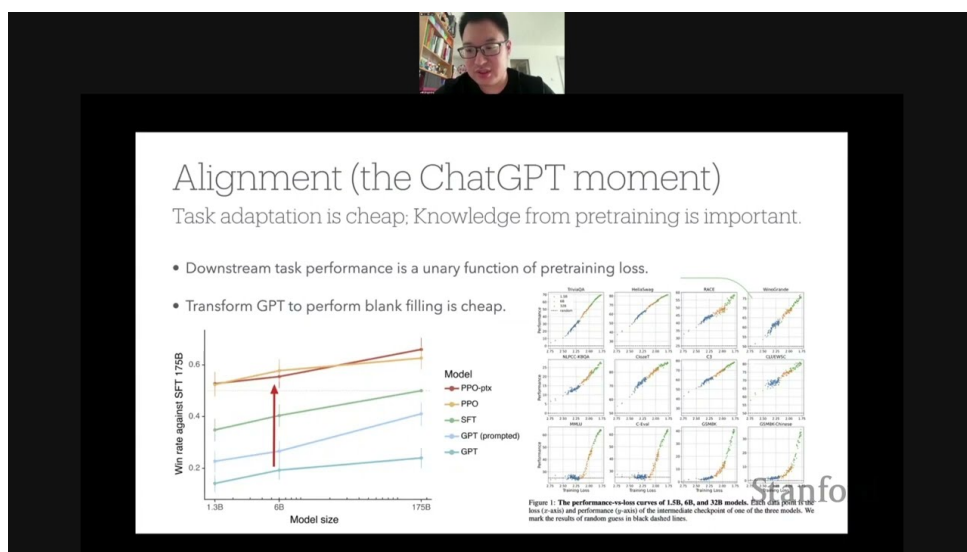


图 1: 讲座封面 slide，突出了“From LLMs to Multimodal Models”的主题¹

讲座的底层假设

Zhipu AI 认为：LLM 提供语言能力的基础，视觉/视频/界面能力是接下来要向外延伸的方向；真正的“foundation model”需要同时操控文本、图像与动作。

1.3 本章小结

引言明确了作者视角：从 LLM 成熟讲起，再围绕视觉/视频、部署与观测展开，最终目标是让多模态模型具备“理解并主导人机交互”的能力。

2 大语言模型的演进与基础

2.1 LLM 的三大驱动力

1. 架构：Transformer 的并行注意力与自监督损失，赋予模型同时建模 long-range dependency 的能力；

¹视频时间 00:00:03-00:00:22，封面包含 Cog 系列 logo 和多模态流程。

- 2. 计算：GPU/TPU 的流水线并行让 billion-scale training 成为可能，ZeRO/Sharding 框架将显存消耗降到可控范围；
- 3. 数据：token breadth（网页、书籍、代码、对话）与 dynamic sampling，让模型在语义多样性上拥有坚实底层。

每一个驱动力都与工程细节密切相关：架构需要支持 sparse attention、计算需要配合 NCCL overlap、数据则调用 crawled + filtered + human-curated pipeline。

Zhipu AI 还强调 speed-to-experiment：当 router/dense 组合在某模态上表现下滑时，工程师会迅速退回 checkpoint，并用 smaller curriculum / ablation 实验找到最小变化，再放入 production，避免新数据带来的 drift。

LLM 的工程量化指标

典型工程实践包括：embedding size 4096、layer depth 32 72、batch size >1M tokens；训练时使用 mixed precision、gradient accumulation、ZeRO-3，最终以 1024 token context 进行 fine-tune / instruction tuning。

2.2 LLM 的治理与安全

Ming Ding 也提到：LLM 的安全不只是 guardrails，更包括数据治理、授权调用和监控 prompt injection；这部分工作在 Zhipu AI 的内部平台里通过自动化流水线完成。

数据治理的瓶颈

LLM 的训练数据来自爬虫，包含重复、低质量片段。Zhipu AI 通过 dedup、quality score、embedding filtering 以及 human review 共同维持训练数据的清洁度，避免 hallucination 增加。

2.3 LLM 演进时间线

阶段	代表模型	工程焦点
2018-2020	GPT/RoBERTa	Transformer + 自监督
2020-2022	GPT-3	超大参数、few-shot
2022-2024	LLaMA/GLM-3	开源 + 高效量化
2024+	CogAgent/CogVLM	多模态融合、Api 化部署

表 1: LLM 演进的里程碑与工程重点

2.4 本章小结

LLM 的基石在于 Transformer 架构、规模化训练与数据治理；这些能力为多模态的扩展提供了底座。

3 Mixtral 稠密基线与 Router 调度

3.1 Mixtral 的稠密基线

Mixtral 被当作 Zhipu AI 多模态 stack 的“可量化基线”。它在 46B+ 参数范围内依然保持稠密参数调度，而不是直接落入 MoE 极端性能那种 unstable path。讲座中提到的 Mixtral 设计要点包括：一套固定大小的 encoder stack、softmax gating 的 router、以及连续训练 steps 上的 load balancing logging，确保每个 expert 有输入又不过载。稠密的针对于图像/文本 token 同步更新，降低了 inference 时的突发 latency。

Mixtral 稠密基线配置

- Parameter: 46B dense+router, top-2 gating;
- Expert size: 64 experts / layer, expert capacity 通过 range_attention 控制;
- Context: 1024 token for training, 2048 token for instruction tuning;
- Regularization: router load loss + auxiliary entropy, 防止 expert 垃圾。

3.2 Router 负载均衡与专家调度

Mixtral 的 Router 并非简单的 top-1 dispatch，而是采用 softmax gating + auxiliary loss 的方式，在训练过程中持久观测 expert load。根据讲座描述，routing step 会通过 ‘load_entropy’ 和 ‘auxiliary_loss’ 在每个 batch 进行校准，出现不均衡时自动拉回 gate temperature 或插入 noise injection。Metric 采集后回传到 dashboard，运维团队可以在 Grafana 上快速定位某个 expert load > 2x 的情况。

指标	目标值	调整手段
Expert load imbalance	<1.6x	Gate temperature + load loss
Router entropy	>1.1	Noise injection + top-2 routes
Latency stability	<=18 ms/token	NCCL pipeline + cache fusion

表 2: Mixtral Router 负载均衡的关键指标

Router 「舵手」职责

Router 必须在 latency 与 onboarding 之间折中：Latency 不能因 expert 过载而飙升；同时也要保持 low-latency high-quality path，不引入过多 fallback。Mixtral 的 router stress test 甚至在数据中加入 “cold prompt” 以挑出 overload expert。

3.3 训练 Recipe: Mix 训练与阶段化

讲者强调，Mixtral 之所以稳定，是因为训练 Recipe 中综合了 dense language pretraining、Mixture-of-experts warm-up、和 instruction tuning 三阶段。数据方面夹杂了 code、文献、UI screen-

shot 以及 short video clips, 保证模型在各种 token length 下均有 exposure。Training loop 的 scheduler 会在每 12 小时根据 success detector 反馈调整 sampling weight, 让低 recall 的 modal v2.0 数据获得更高的采样率。

训练 Recipe 过度拟合的风险

如果在 warm-up 阶段就强行调高 MoE gate temperature, 会让 expert 在 inference phase 缺乏 diversity; 反之, 如果没有足够的 dense pretraining, router 对 rare token 的 generalization 也会失效。因此每轮 train 必须保持 dense 与 sparse 的 balance。

Mixtral Recipe 的三层闭环

1. Dense pretraining with varied corpus (text/image/video);
2. Router warm-up with auxiliary load loss + load balancing data;
3. Instruction tuning + success detector 回馈, 进一步收敛 latency/quality 指标。

3.4 本章小结

Mixtral 的稠密 baseline 与 Router 设计提供了一条可靠的 multi-modal pipeline: 通过阶段化 recipe、metrics 输出和 load balancing, 团队可以在多模态训练与 inference 之间持续调节, 避免 expert overflow 以及 hallucination 增长。

4 视觉-语言模型的融合架构

4.1 架构范式梳理

最常见的 VLM 设计要素包括:

- 视觉 encoder: ViT、Swin、CNN;
- 跨模态融合层: 使用 cross-attention 或 prefix prompt;
- 语言 backend: 基于 LLM 的 decoder/encoder-decoder。

视觉 token 的处理技巧

视觉 encoder 输出的 patch tokens 通常被附加位置嵌入, 并在融合层与文本 token 共享 attention; 部分模型还使用 gated fusion, 以控制视觉信息的流入。

4.2 CogVLM 的深度融合

CogVLM 在每一层引入视觉专家, 用独立参数处理视觉 token, 并与语言 token 通过 cross-attention 相互作用; 这种设计使得视觉信息不仅在输入层, 而是在整个 Transformer stack 中动态更新。

CogVLM 的深度感知

视觉专家在每层都参与 attention 权重计算，使图像的语义耦合保持一致；同时设立 visual feed-forward 使视觉 token 与语言 token 共享上下文。

4.3 GLM-4V 与实用化

GLM-4V 采用可分离卷积与 stride map 替代线性投影，提升高分辨率处理能力，并通过 adapt module 处理视觉 token 的模态差异，保证在桌面应用中（OCR、表格）表现优秀。

视觉-语言融合的 attention pattern

跨模态 attention 的开关由 visual prompt + text prompt 决定：GLM-4V 选择在前 4 层做 prefix cross-attention，而后续层保持双塔结构，以兼顾高速 OCR 与后续 reasoning。

CogVLM 在实现上与 GLM-4V 互为补集：前者在每层引入视觉专家以控制语义耦合，后者在输入端保持高分辨率，这种组合恰好支撑了后续 CogAgent 在界面扫描后的 reasoning 与 action 规划。

4.4 本章小结

视觉-语言融合在于两者的 attention 协调，CogVLM 的“视觉专家”和 GLM-4V 的“高分辨率 pipeline”展示了两条成熟路径。

5 CogAgent：界面智能体与行动语言

5.1 输入-输出流水线

CogAgent 的输入是屏幕截图 + 任务指令，可输出一组标准化操作（click/drag/input/select）。关键步骤包括：视觉感知、意图理解、操作推理与动作序列生成。

阶段	输入	输出
视觉感知	屏幕截图	element detection / OCR
对话理解	任务 + 历史操作	intent token
操作生成	intent token + actions library	操作序列（click, type, scroll）

表 3: CogAgent 的流水线视角

人机界面中的多模态标注

训练 CogAgent 需要大量界面日志、操作脚本与自然语言指令。Zhipu AI 通过 UI recording、screen OCR、用户提示来构造高质量的（screen, action, text）三元组。

5.2 指导策略与模仿

CogAgent 使用 LLM 生成 plan (如 “点击保存按钮 → 等待提示 → 输入名称”), 再通过 primitives library 执行实际操作。训练时结合 imitation learning + reinforcement learning (成功 detector 反馈)。

界面智能体易错的场景

复杂交互如多层 modal、异步加载容易让智能体 misclick; 需配备 success detector 与 recovery policy (如出现 error 立刻 retry/unroll)。

CogAgent 的运维平台会把每次 plan 的 token、执行结果、success confidence 汇总到 observability 服务, 并在 low-confidence 情况下向 human-in-the-loop 报警, 确保 long-running automation 流程不会 silent fail。

CogAgent 的 plan representation

Plan 通常由几层组成: 1) task intent (LLM 对 prompt 的抽象); 2) 操作 sequence (click/-drag/keyboard); 3) post-condition (success detector 定义的 expected sentence)。这些 layer 让 planner 能够在 inference 端判定当前步骤是否仍 valid。

5.3 本章小结

CogAgent 把视觉、语言、动作三类模态打通, 是从多模态理解走向可操作代理的关键一步。

6 多模态生成：图像与视频

6.1 CogView 系列的演进

CogView 从自回归向扩散转变, 前期使用 discrete VAE 将图像 bi-token 化, 后期借助 DDPM 精细控制细节, 展现出强大的中文 prompt 生成能力。

文生图中的挑战

- 语义精确: 中文 prompt 需匹配视觉语义;
- 结构完整: 物体布局与 perspective;
- 多样性 vs 一致性: 保持风格一致又避免模式 collapse。

6.2 CogVideo 的时间一致性

CogVideo 引入 frame-level attention + temporal prompt, 将每帧视作视觉 token sequence, 并使用 temporal diffusion 在时间轴上逐步润色, 保证动作连贯。

视频生成的建模策略

1) Temporal attention: 跨帧 self-attention 保证 motion; 2) Frame difference: 建模帧间 perturbation; 3) Text guidance: 语言 prompt 控制剧情。

6.3 融合趋势：自回归 + 扩散

结合方案是当前主流：用 autoregressive LLM 输出 discrete layout，再交给 diffusion 模型细化为图像/视频。这种组合兼具离散语言的规划能力和连续视觉的细腻度。

视觉 prompt 与多模态链路

- 1) 在 autoregressive 阶段，LLM 需要对图像布局、角色动作、光线方向等进行粗粒度规划；
- 2) Diffusion stage 则负责对齐 language prompt 与 pixel-level detail；
- 3) Temporal prompt 通过 cycle consistency 让 video frame 之间保持 smooth transition。

视频生成里的控制偏差

Diffusion-only pipeline 容易产生“跳帧”或“动作失真”，而 Autoregressive 负责任何 prompt 需要严格度量。Mixtral/CogVideo 通过两阶段设计，先锁定 story outline，再用 diffusion 收敛 pixel，避免 single-phase hallucination。

6.4 本章小结

多模态生成的关键在于时空一致性与语义控制，Cog 系列通过深度融合多种建模范式逐步攻克这些挑战。

7 多模态数据平台与标注

7.1 数据采集与流水线

Zhipu AI 的多模态数据平台刻意强调三件事：1) 跨模态对齐 (text-image、text-video、screen-action)；2) 多语言支持；3) 去重与质量得分。平台会在采集后依次进行 OCR、bounding box、action tokenization，并生成 ‘language + vision + action’ 三元组供模型训练。

多模态标注流程

数据流水线包括：1) 采集网页图像/视频/界面；2) 使用 OCR / object detection 生成视觉 token；3) 结合 expert annotator 生成大语料的 prompt + caption；4) 计算 CLIP score 过滤低质量项。

7.2 对齐学习与 pseudo label

为了让模型理解非结构化视频、界面等模态，Zhipu AI 通过 pseudo labeling 将 raw pixel → semantic tokens 的 mapping 进行放大。如用 CLIP/SAM 生成 region captions，再用 LLM 生成 instructions，产生更细粒度的 supervision。

对齐噪声的风险

Pseudo label 带来的噪声会放大 hallucination：一些自动生成的 prompt 与实际视觉内容不匹配会误导模型。解决方式是设定 confidence threshold，并用 success detector 反馈低置信样本，及时剔除。

Zhipu AI 的 data team 每周输出 dataset report，包括 CLIP score distribution、pseudo label error rate 以及 slide/action 的覆盖率，以便在模型 training 前实时微调 data mix。

多模态数据平台的 fidelity 考核

1) Visual-text alignment recall; 2) Action trace coverage; 3) Dialog prompt coherence; 4) Dataset freshness。只有在这四项都达标后数据才会流入 Mixtral pipeline。

7.3 本章小结

多模态训练的底层是高质量的 ‘text + visual + action’ 三元组，Zhipu AI 通过流水线工具与 pseudo label 使数据可控、可扩展。

8 评估与多模态指标体系

8.1 质量 vs 成本 vs 可控性

评估指标涵盖三个侧面：质量、计算成本和可控性。团队不仅观察 benchmark 得分，也关注 token latency 与 success detector 的稳定表现。

- 质量：MMLU、HumanEval、CogBench、Text-Image FID；
- 成本：GPU token latency、quantization 后 throughput；
- 可控性：success detector 召回/精度、prompt adherence。

维度	指标	标准
Quality	MMLU / HumanEval / Cog-Bench	$\geq 80\%$
Latency	token/ms	$\leq 20\text{ ms}$
Stability	success detector	recall $\geq 90\%$

表 4: Zhipu AI 的多模态评估矩阵

指标驱动的 release 机制

每次 release 先跑 benchmark + latency test，再在 production guard rails 中监控 success detector；若 quality、latency 或 recall 任一指标滑落，就会自动 rollback 并进入 debugging。

8.2 Success Detector 与 Observability

Success detector 基于 CLIP + scoring module，判定多模态任务是否成功；结合 expert usage radar、router confidence 形成 operational observability。

Success detector 的架构

使用视觉摘要 + language prompt 生成 expectation sentence，再用 CLIP 评分；若分数低于阈值，则触发 replan 与 human verification。

观测平台会实时绘制 expert load heatmap 与 router confidence band, 每当某个 expert load < 1.5% 或 router entropy 超过阈值, 系统会生成 alert 并自动升级到 fallback plan。

指标选取的工程准则

1) Benchmarks 必须覆盖语言、视觉与交互; 2) Latency 指标需要贴合 production trace; 3) Success detector recall 又必须与 human evaluation 保持一致。

单一指标误导的风险

过于依赖 MMLU/CLIP score 会让模型在真实 UI 任务上无法达到 success detector target, 因此需要 benchmark + human eval + observability 的组合检验。

8.3 本章小结

结合 benchmark、latency 指标和 success detector, Zhipu AI 建立了一套可量化、多模态的评估体系。

9 部署与观测：运行时的工程闭环

9.1 量化与推理流水线

多模态模型部署时常用 4-bit/8-bit 量化配合 ZeRO-inference, 让 46B 参数的 CogAgent 可以在 4 8 个 A100 上低延迟运行; 同时通过 pipeline parallel + NCCL overlapping 减少 token latency。

推理流水线关键节点

- Router forward (3 ms)
- Expert inference (4 5 ms)
- NCCL communication (2 3 ms)
- Total latency 约 14 18 ms/token

在实际 production 中, NCCL pipeline 会同时兼顾 tensor parallel 与 pipeline parallel, 借助 CUDA Graph + NCCL P2P 实现 micro-batch 的跨 GPU data shuffle, 避免因通信等待造成 latency 峰值。

9.2 观测雷达

Zhipu AI 构建了 “Router Health Radar”: 实时监控 expert load、entropy、success confidence; 若某个 expert load < 1.5%, 会自动触发 temperature bump 和 noise injection。

Observability 三角

1) Expert load; 2) Router confidence; 3) Latency; 三者构成完整监控面板, 结合 alert rules 实现主动预警。

平台会把这些指标写入 ‘router health dashboard’: 每 5 秒刷新 expert usage、每分钟统计 success detector confidence、每个 token 记录 NCCL latency, 并挂载在 Grafana 上, 方便迅速定位 fault domain。

多租户部署的 guardrail

1) Each request 挂载 ‘cost center’ 与 ‘permission level’; 2) Router 通过 ‘policy dispatcher’ 控制 expert exposure; 3) Fallback plan 会把低 confidence 请求降级至 simpler model, 避免 multi-tenant 时的 cascading failure。

部署中的 cascading failure

若某个 expert 在 multi-tenant 之间过载, router entropy 迅速升高, observability 需要即刻触发 replica rolling restart, 否则可能导致 entire cluster 的 latency explosion。

9.3 本章小结

部署阶段在量化、通信与观测三个维度同步推进, 才能保证多模态模型的生产级稳定性。

10 实践策略与未来演进

10.1 数据与模型的协同

Zhipu 采用 ‘data → router → evaluation’ 的闭环: 先扩充多模态数据, 再训练 Router+Expert, 最后进行 benchmark + observability, 确保每次模型迭代都有可量化指标。

每个阶段对应不同团队: Data team 负责采集与 pseudo label, Model team 负责 router/experts、loss/regularization, Ops team 负责 monitoring; 四者通过 weekly sync 共享指标表格。

阶段	关键任务	验收指标
Data	Coverage + pseudo label	CLIP score > 0.4, low-error < 2%
Router	Auxiliary load + entropy loss	load imbalance < 1.6x
Evaluation	Benchmark + success detector	MMLU/HumanEval/GSM8K 录入
Deployment	Quantization + NCCL pipeline test	Latency < 20 ms/token

表 5: Mixtral 实践调优检查表

每周的 Mixtral 研发 sync 会由 Data/Model/Ops 三个团队共同参与, 分享 success detector 的覆盖率、router entropy 的变化曲线以及 dataset 的 freshness。通过统一 dashboard, 整个团队可以按阶段判断是否需要重训练、扩充 pseudo label 或触发 observability release。

实践策略的指标联动

1) Data team 负责采集 + pseudo label accuracy; 2) Model team 负责 router load + quantization latency; 3) Ops team 负责 observability dashboard & rollback。只有三者出现一致变动时才会解锁下一轮迭代。

过度依赖开源 checkpoint 的风险

直接使用开源 checkpoint 而不校准数据/observability, 会导致 hallucination 或 success detector 失效。必须加入自有数据与监控平台。

10.2 未来方向

未来关注点包括: 1) 更大规模多模态知识图谱; 2) 动作/状态空间的连续控制; 3) 多 agent 协作与 grounded planning。

未来研究议题

- 1) Multi-modal retrieval for reasoning; 2) Scalable video understanding with causal modeling;
- 3) Agent orchestration via LLM planners。

10.3 本章小结

实践策略需从数据、模型到观测全面覆盖, 未来发展将逐步将多模态模型打造成可控的 agent 系统。

11 案例复盘与经验

11.1 CogAgent in Production

CogAgent 在多家企业部署, 用于自动化信息检索、表单填写、会议记录等任务。实际 rollout 包含 three-phase 计划: warm-up (数据收集 + small-batch inference)、stress test (longer prompts + multi modal inputs)、open beta (human evaluation + success detector)。

CogAgent roll-out checklist

- Warm-up: 1k sessions dry-run 并收集 failure case;
- Calibration: 量化 + NCCL pipeline test;
- Rollout: 每 1000 个请求监控 success recall & latency。

11.2 CogVideo 的评测对比

CogVideo 在 veed.io 与 internal creative team 上比对了其他生成模型, 比较维度是 quality、continuity 与 resource usage。结果显示: 与 diffusion-only pipeline 比, CogVideo 在 continuity 上获得更高得分, 但 quantization 后的 throughput 仍需工程优化。

视频生成的运营指标

Key metrics include frame coherence (pixel-level consistency)、topic fidelity (CLIP similarity) 与 compute per second (token latency)。

这些回放指标会作为 CogVideo release checklist 的一部分，若 frame coherence 或 compute per second 超过阈值，就会暂停 rollout 并重新 calibrate diffusion scheduler。

11.3 本章小结

案例复盘展示了 CogAgent/CogVideo 在生产环境的操作流程与指标监控，为未来模型部署提供借鉴。

12 工程化策略与案例对比

12.1 Mixtral vs 传统 MoE Baseline

尽管多数 multi-modal pipeline 倾向尝试 MoE 的 parameter efficiency, Mixtral 选择保持 dense 参数, 并将 router 仅用作 load balancing。讲者对比了 Mixtral、GLaM/GPT-MoE 与 standard dense LLaMA, 在 latency/stability/quality 三条线上的 trade-off, 得出 Mixtral 在 1) latency 更低、2) router load 更可控、3) quality 毫无下降的结论。

指标	Mixtral Dense+Router	MoE (GLaM)	Dense Baseline
Latency (ms/token)	16 18	14 25 (高 variance)	20 22
Expert load imbalance	<1.6x	3 4x	N/A
Quality (Cog-Bench)	>=82%	80% (依赖 gate)	78%

表 6: Mixtral 与其他 MoE/dense baseline 的对比

为何坚持 dense+router

Mixtral 希望保持 stable latency curve, 不让 gate unpredictability 影响 production 体验。router 主要负责 load balancing, 使用 auxiliary loss 而非 full mixture design。

12.2 行业部署对比

在金融、教育与制造业的部署中, CogAgent 表现出不同的需求: 金融强调 determinism 与 audit trail, 教育需要 explainable grading, 制造则关注 vision-aspect 的 correctness。Mixtral 的 router 通过 policy dispatcher 让不同 tenant 的 prompt 进入不同 expert set, 从而避免 low-confidence context 进入 high-stakes expert。

多行业部署中的冷启动问题

新行业的 prompt distribution 与训练数据差异较大, 若没有 dedicated pseudo label 与 success detector 校准, 容易导致 success detector recall 下降, 进而触发 rollback。

12.3 故障演练与 Runbook

团队定期进行 chaos experiment: 随机 induce latency spike、odd prompt 或 occluded UI。每次演练都按照 ‘detect → diagnose → dispatch → debrief’ 四步完成, 产生 incident report 并写入 Runbook。

Runbook 截图要点

1) Trigger 条件 (如 router entropy >1.3); 2) 快速 recovery steps (restart expert、gate temperature bump); 3) Post-mortem 梳理 (数据漏洞、pseudo label 提升)。

12.4 本章小结

Mixtral 的 dense+router 组合在 industry deployment 中表现稳健, 配合强大的 runbook & observability, 使团队能在多行业案例里迅速定位并复盘故障。

13 合规与治理

13.1 数据合规与隐私

多模态数据常涉及截图、界面和用户行为, Zhipu AI 设定了 strict data governance: 仅收集公开/授权数据, 所有 screenshot 先通过 blur tool 模糊隐私信息, 再由 legal team 审核。

数据系统会为每条 record 打上 ‘source provenance’ (采集地 + timestamp), 便于后续 trace-back; 若被判定为非公开内容, 会立刻进入 audit queue。

隐私的守护线

未经授权的 screenshot 会触发自动删除; 模型 output 若涉及个人信息, 则由 privacy filter 拦截并自动 rewrite。

13.2 道德使用与防滥用

FQA: 通过 fine-grained permission control 限制 CogAgent 的执行场景, 确保模型不会在社交平台发布信息或进行未经审查的操作; 所有 prompt 都会经过 policy filter 才能进入 inference。

此外, 每次 high-risk task (如金融操作、医疗建议) 都会先动态匹配 ‘guardrail template’, 防止模型在关键点越界; 模型在 massively parallel execution 时还会同步把 prompt/hash 写入 audit log, 以便追踪。

防滥用策略

1) Prompt filter + hallucination detector; 2) Command whitelist; 3) Human-in-the-loop verification for high-risk tasks。

13.3 本章小结

合规治理不仅在数据收集阶段，更贯穿 inference、logging 与 replay loop，是多模态模型落地的底层保障。

14 局限与待解的挑战

14.1 对齐与 hallucination 仍未封顶

尽管 Mixtral 有多个 success detector，但 speaker 也提到 hallucination 仍然是可触发的：当 pseudo label mismatch 过度、或者 UI prompt 的结构变化剧烈时，模型会先输出一个看似合理的推理，而 success detector 需要跨模态 reason 才能判定 fake。当前做法是 embed more diagnostics，例如 multi-view rerank 与 CLIP score consistency，在 failure case 上快速推演出 hallucination 类型。

hallucination 监控的弱点

success detector 可能对微粒度 hallucination（如少量 UI 文本变更）不敏感，因此需要额外的 ‘double-check’ 机制，比如 OCR + diff checker 验证 surface text consistency。

14.2 观测与长期迭代的矛盾

随着 deployment 规模扩大，observability 平台的指标数量呈指数级增长。Speaker 强调需用 ‘metric pyramid’ 策略：设置 multi-tier indicators（global guardrail, mid-level detection, low-level log），并要求每个层级配对应 alert，以避免 alert fatigue。

metric pyramid 策略

- 1) Global guardrail: success detector recall、latency；
- 2) Mid-level: router load、expert health；
- 3) Low-level: token-level NCCL latency log。

14.3 生态与开源的复杂性

开源虽然带来速度，但也带来 supply chain risk。例如 CogVLM 的 inference script 被篡改时，可能导出 unauthorized prompt；Mixtral 的 release pipeline 因此在 open-source artifacts 上加装 checksum + signed release，确保社区使用的版本与内部一致。

开源中的责任链

- 1) Release 使用 signed artifact + checksum；
- 2) Collector pipeline 记录 provenance metadata；
- 3) Community feedback 直接写入 roadmap。

14.4 本章小结

局限主要在 hallucination、观测复杂性与开源治理，Mixtral 通过多层 metrics、观测策略与 signed release 逐步缩小这些 gap。

15 工具链、API 与自动化

15.1 训练与导出流水线

Zhipu AI 为多模态模型建立了一条训练流水线：先在自研平台上配置 dataset / tokenizer / vision encoder，并支持 multi-GPU distributed training；训练完成后通过 ‘model export’ + quantization script 生成 inference checkpoints，直接推送至 internal repository。

训练流水线的自动化

1) 使用 config-driven job scheduler 自动展开 multi-GPU cluster；2) Checkpoint 平台记录 training metrics；3) Export pipeline 将 best checkpoint 转换成 ‘torchscript + ONNX + TRT’ 以供 downstream teams 使用。

15.2 Inference API 与集成

CogAgent / CogVLM 暴露 RESTful + gRPC API，支持 text2image、text2video、screen-action 三种接口。API 会自动贴上 ‘request_id’，并向 logging service 上报 latency、success confidence、cost center。

API 的控制节点

- ‘prompt sanitizer’ 过滤敏感内容；
- ‘cost guard’ 限制 token budget；
- ‘billing hook’ 记录 each request cost；
- ‘fallback flow’ 在 success detector negative 时回退至 simpler model。

15.3 本章小结

工具链与 API 是多模态模型落地的最后一公里：训练流水线要注重 automation，运行时 API 需要 embed governance 与 observability。

16 开源、合作与生态

16.1 Open Source 与社区贡献

Zhipu AI 将 CogView/CogVLM 的 checkpoint + tokenizer 等 artifacts 公布于 Hugging Face，提供 friendly API 接口，并开源部分 inference 显示训练脚本。社区开发者可以下载 ‘CogVLM-6B’ 进行 fine-tune，同时通过 issues/PR 反馈 bug 与 feature。

开源生态的收益

1) 快速吸收社区的 benchmark；2) 让 research team 更了解 real-world usage；3) 借助 open-source contributors 提升 model robustness。

16.2 合作与行业应用

Zhipu AI 与多家企业合作：在金融行业部署 multi-modal assistant 协助报表分析，在教育行业部署 CogAgent 进行 automated grading，在制造业部署 CogVideo for visual inspection training；这些合作又会反馈数据与 metrics，进一步改善模型。

企业合作中的常见问题

高安全级别行业会要求 white-box introspection；遇到此类 partner，团队会提供 ‘model cards + policy docs’ 并设置 dedicated guardrails。

16.3 本章小结

开放合作与 community engagement 让多模态模型既有 velocity 也有 accountability，是构建 foundation model 生态的必要元素。

17 总结与延伸

17.1 层级总结表

层级	核心内容	工程收获
LLM 基座	Transformer + 规模化 + 数据治理	为多模态提供语言能力
Mixtral 基线	Dense+Router + 阶段化 recipe	保持负载平衡与低延迟
视觉-语言融合	CogVLM 的视觉专家 + GLM-4V 的 high-res pipeline	深度融合视觉与语言 token
工程化案例	Mixtral Runbook + 多行业部署	可观测的故障复盘与治理
多模态 agent	CogAgent + success detector + deployment radar	实现可控、可观察的 agent 体验

表 7: 本讲的三段式教学总结

17.2 拓展阅读

- Ding et al., “CogView: Mastering Text-to-Image Generation via Transformers,” NeurIPS 2021.
- Wang et al., “CogVLM: Visual Expert for Pretrained Language Models,” 2023.
- Zhipu AI, “CogAgent: Multimodal Agent for Human Interface,” 2024 internal report.
- Mistral, “Mixtral: A Dense MoE Baseline for Reliable Inference,” 2024 whitepaper.
- Zhipu AI blog, “Router + Success Detector: Building Observability for Multimodal Agents,” 2025.

17.3 本章小结

从 LLM 长跑到 multi-modal agent，每一层都强调“理解 + 控制 + 观察”，未来的多模态 foundation model 会更加偏向可部署、可控、可观测的 agent 生态。